



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Department of Computer Science
Faculty of Computing

SECP3623 – DATABASE PROGRAMMING

(Assignment 1)

Programme	: Bachelor of Computer Science (Data Engineering)
Subject Code	: SECP3623
Subject Name	: DATABASE PROGRAMMING
Session-Sem	: 2025/2026-01

Prepared by : 1) Muhammad Afiq Danish bin Mohd Hazni (A23CS0118)
2) Muhammad Syahmi Faris bin Rusli (A23CS0138)

Section : 02

Lecturer : Dr. Shamini A/P Raja Kumaran

Task 1 - Database Creation and Integrity Constraints

```
CREATE DATABASE hostel_mgmt_Afiq;  
USE hostel_mgmt_Afiq;
```

Database creation

```
CREATE TABLE room_types(  
    type_id INT PRIMARY KEY,  
    type_name VARCHAR(30) NOT NULL,  
    rent DECIMAL(6, 2) NOT NULL,  
    deposit DECIMAL(6, 2) NOT NULL,  
    capacity INT NOT NULL  
);
```

Creation of *room_types* table

```
CREATE TABLE rooms(  
    room_id INT PRIMARY KEY,  
    room_no VARCHAR(10) NOT NULL,  
    floor_no INT NOT NULL,  
    is_occupied BOOLEAN DEFAULT FALSE,  
    type_id INT,  
    CONSTRAINT fk_type_id  
        FOREIGN KEY(type_id) REFERENCES room_types(type_id)  
);
```

Creation of *rooms* table

```
CREATE TABLE students(  
    student_id INT PRIMARY KEY,  
    fname VARCHAR(50) NOT NULL,  
    lname VARCHAR(50) NOT NULL,  
    status VARCHAR(30) NOT NULL,  
    checkin_date DATE NOT NULL,  
    room_id INT,  
    CONSTRAINT fk_room_id  
        FOREIGN KEY(room_id) REFERENCES rooms(room_id)  
);
```

Creation of *students* table

```

CREATE TABLE maintenance(
    maint_id INT PRIMARY KEY,
    issue_desc TEXT NOT NULL,
    severity VARCHAR(20) NOT NULL,
    status VARCHAR(20) NOT NULL,
    reported_on DATE NOT NULL,
    resolved_on DATE,
    room_id INT,
    CONSTRAINT fk_room_id2
        FOREIGN KEY(room_id) REFERENCES rooms(room_id)
);

```

Creation of *maintenance* table

```

CREATE TABLE payments(
    payment_id INT PRIMARY KEY,
    amount DECIMAL(6, 2) NOT NULL,
    paid_on DATE NOT NULL,
    method VARCHAR(20) NOT NULL,
    note VARCHAR(70) NOT NULL,
    student_id INT,
    CONSTRAINT fk_student_id
        FOREIGN KEY(student_id) REFERENCES students(student_id)
);

```

Creation of *payments* table

```

ALTER TABLE students
    ADD COLUMN email VARCHAR(100) UNIQUE;

```

Column addition (email) in students table with unique constraint

```

CREATE TABLE test(
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL,
    phoneNumber VARCHAR(20) NOT NULL
);

DROP TABLE IF EXISTS test;

```

Creation of temporary test table and deletion of the table

Task 2 - Data Manipulation and Filtering

Query:

```
INSERT INTO room_types(type_id, type_name, rent, deposit, capacity)
VALUES
(1, 'Single', '600.00', '300.00', 1),
(2, 'Double', '400.00', '200.00', 2),
(3, 'Premium', '900.00', '450.00', 1),
(4, 'Family', '1000.00', '500.00', 4),
(5, 'Twin Share', '350.00', '175.00', 2),
(6, 'Deluxe Single', '750.00', '375.00', 1),
(7, 'Triple', '300.00', '150.00', 3),
(8, 'Executive Suite', '1200.00', '600.00', 1),
(9, 'Budget Double', '280.00', '140.00', 2),
(10, 'Studio', '650.00', '325.00', 1);
```

Data insertion into *room_types* table

Result:

type_id	type_name	rent	deposit	capacity
1	Single	600.00	300.00	1
2	Double	400.00	200.00	2
3	Premium	900.00	450.00	1
4	Family	1000.00	500.00	4
5	Twin Share	350.00	175.00	2
6	Deluxe Single	750.00	375.00	1
7	Triple	300.00	150.00	3
8	Executive Suite	1200.00	600.00	1
9	Budget Double	280.00	140.00	2
10	Studio	650.00	325.00	1
NULL	NULL	NULL	NULL	NULL

Query:

```
INSERT INTO rooms(room_id, room_no, floor_no, type_id)
VALUES
(101, 'A101', 4, 1),
(102, 'B101', 3, 2),
(103, 'C101', 2, 3),
(104, 'D101', 1, 4),
(105, 'E101', 5, 5),
(106, 'F101', 6, 6),
(107, 'G101', 1, 7),
(108, 'H101', 2, 8),
(109, 'J101', 3, 9),
(110, 'K101', 3, 10);
```

Data insertion into *rooms* table

Result:

room_id	room_no	floor_no	is_occupied	type_id
101	A101	4	0	1
102	B101	3	0	2
103	C101	2	0	3
104	D101	1	0	4
105	E101	5	0	5
106	F101	6	0	6
107	G101	1	0	7
108	H101	2	0	8
109	J101	3	0	9
110	K101	3	0	10
NULL	NULL	NULL	NULL	NULL

Query:

```
INSERT INTO students(student_id, fname, lname, status, checkin_date, room_id)
VALUES
('0118', 'Afiq', 'Danish', 'ACTIVE', '2025-01-10', 101),
('0138', 'Syahmi', 'Faris', 'ACTIVE', '2025-01-10', 102),
('0205', 'Afif', 'Shaqir', 'NON_ACTIVE', '2024-01-10', 103),
('0116', 'Adam', 'Razali', 'NON_ACTIVE', '2024-01-09', 104),
('0100', 'Naim', 'Abdullah', 'ACTIVE', '2025-01-09', 105),
('0185', 'Mukhritz', 'Iman', 'ACTIVE', '2025-01-11', 106),
('0218', 'Afiq', 'Danial', 'ACTIVE', '2025-01-10', 107),
('0098', 'ALif', 'Fathi', 'NON_ACTIVE', '2024-01-08', 108),
('0170', 'Mikail', 'Aisy', 'ACTIVE', '2025-01-12', 109),
('0075', 'Rizwan', 'Hakim', 'NON_ACTIVE', '2023-10-10', 110);
```

Data insertion into *students* table

Result:

student_id	fname	lname	status	checkin_date	room_id
75	Rizwan	Hakim	NON_ACTIVE	2023-10-10	110
98	ALif	Fathi	NON_ACTIVE	2024-01-08	108
100	Naim	Abdullah	ACTIVE	2025-01-09	105
116	Adam	Razali	NON_ACTIVE	2024-01-09	104
118	Afiq	Danish	ACTIVE	2025-01-10	101
138	Syahmi	Faris	ACTIVE	2025-01-10	102
170	Mikail	Aisy	ACTIVE	2025-01-12	109
185	Mukhritz	Iman	ACTIVE	2025-01-11	106
205	Afif	Shaqir	NON_ACTIVE	2024-01-10	103
218	Afiq	Danial	ACTIVE	2025-01-10	107
NULL	NULL	NULL	NULL	NULL	NULL

Query:

```
INSERT INTO maintenance(maint_id, issue_desc, severity, status, reported_on, resolved_on, room_id)
VALUES
(1, 'Aircond not working', 'HIGH', 'RESOLVED', '2025-02-01', '2025-02-05', 101),
(2, 'Light bulb broken', 'LOW', 'RESOLVED', '2025-02-15', '2025-02-19', 102),
(3, 'Leaking Ceiling', 'MEDIUM', 'OPEN', '2025-02-15', NULL, 103),
(4, 'Broken Bed', 'HIGH', 'RESOLVED', '2024-03-28', '2024-03-29', 104),
(5, 'Light bulb broken', 'MEDIUM', 'RESOLVED', '2025-04-16', '2025-04-20', 105),
(6, 'Leaking Ceiling', 'LOW', 'OPEN', '2025-05-01', NULL, 106),
(7, 'Electric switch not working', 'LOW', 'OPEN', '2025-04-30', NULL, 107),
(8, 'Broken Furniture', 'HIGH', 'RESOLVED', '2024-09-02', '2024-09-10', 108),
(9, 'Aircond not working', 'MEDIUM', 'RESOLVED', '2025-04-29', '2024-05-05', 110),
(10, 'Broken Bed', 'HIGH', 'OPEN', '2025-04-30', NULL, 102);
```

Data insertion into *maintenance* table

Result:

maint_id	issue_desc	severity	status	reported_on	resolved_on	room_id
1	Aircond not working	HIGH	RESOLVED	2025-02-01	2025-02-05	101
2	Light bulb broken	LOW	RESOLVED	2025-02-15	2025-02-19	102
3	Leaking Ceiling	MEDIUM	OPEN	2025-02-15	NULL	103
4	Broken Bed	HIGH	RESOLVED	2024-03-28	2024-03-29	104
5	Light bulb broken	MEDIUM	RESOLVED	2025-04-16	2025-04-20	105
6	Leaking Ceiling	LOW	OPEN	2025-05-01	NULL	106
7	Electric switch not working	LOW	OPEN	2025-04-30	NULL	107
8	Broken Furniture	HIGH	RESOLVED	2024-09-02	2024-09-10	108
9	Aircond not working	MEDIUM	RESOLVED	2025-04-29	2024-05-05	110
10	Broken Bed	HIGH	OPEN	2025-04-30	NULL	102
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Query:

```
INSERT INTO payments(payment_id, amount, paid_on, method, note, student_id)
VALUES
(1, 600.00, '2024-12-01', 'CASH', 'December Rent', '0118'),
(2, 200.00, '2024-12-01', 'CARD', 'December Deposit', '0138'),
(3, 900.00, '2024-12-05', 'FPX', 'December Rent', '0205'),
(4, 1000.00, '2024-12-10', 'TNG', 'December Rent', '0116'),
(5, 1200.00, '2024-12-03', 'CASH', 'August Rent', '0098'),
(6, 325.00, '2024-12-01', 'TNG', 'September Deposit', '0075'),
(7, 280.00, '2025-01-05', 'CARD', 'January Rent', '0170'),
(8, 350.00, '2025-01-06', 'FPX', 'January Rent', '0100'),
(9, 375.00, '2025-03-01', 'CASH', 'March Deposit', '0185'),
(10, 300.00, '2025-02-02', 'FPX', 'February Rent', '0218');
```

Data insertion into *payments* table

Result:

payment_id	amount	paid_on	method	note	student_id
1	600.00	2024-12-01	CASH	December Rent	118
2	200.00	2024-12-01	CARD	December Deposit	138
3	900.00	2024-12-05	FPX	December Rent	205
4	1000.00	2024-12-10	TNG	December Rent	116
5	1200.00	2024-12-03	CASH	August Rent	98
6	325.00	2024-12-01	TNG	September Deposit	75
7	280.00	2025-01-05	CARD	January Rent	170
8	350.00	2025-01-06	FPX	January Rent	100
9	375.00	2025-03-01	CASH	March Deposit	185
10	300.00	2025-02-02	FPX	February Rent	218
NULL	NULL	NULL	NULL	NULL	NULL

Query:

```
UPDATE rooms
  SET is_occupied = TRUE
  WHERE room_id IN (
    SELECT room_id FROM students WHERE status = 'ACTIVE');

SELECT * FROM rooms;
```

Updating is_occupied column in *rooms* table

Result:

room_id	room_no	floor_no	is_occupied	type_id
101	A101	4	1	1
102	B101	3	1	2
103	C101	2	0	3
104	D101	1	0	4
105	E101	5	1	5
106	F101	6	1	6
107	G101	1	1	7
108	H101	2	0	8
109	J101	3	1	9
110	K101	3	0	10
NULL	NULL	NULL	NULL	NULL

Query:

```
DELETE FROM maintenance
WHERE status = 'RESOLVED' AND DATEDIFF(CURDATE(), reported_on) > 60;

SELECT * FROM maintenance;
```

Removal of records from *maintenance* table which already resolved and reported date is older than 60 days

Result:

maint_id	issue_desc	severity	status	reported_on	resolved_on	room_id
3	Leaking Ceiling	MEDIUM	OPEN	2025-02-15	NULL	103
6	Leaking Ceiling	LOW	OPEN	2025-05-01	NULL	106
7	Electric switch not working	LOW	OPEN	2025-04-30	NULL	107
10	Broken Bed	HIGH	OPEN	2025-04-30	NULL	102
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Query:

```
SELECT * FROM room_types
WHERE rent BETWEEN 400 AND 800;
```

Display room types that have the rent value between RM400 and RM800

Result:

type_id	type_name	rent	deposit	capacity
1	Single	600.00	300.00	1
2	Double	400.00	200.00	2
6	Deluxe Single	750.00	375.00	1
10	Studio	650.00	325.00	1
NULL	NULL	NULL	NULL	NULL

Query:

```
SELECT * FROM students
WHERE fname LIKE 'A%';
```

Display students whose name starts with the letter 'A'

Result:

student_id	fname	lname	status	checkin_date	room_id
98	Alif	Fathi	NON_ACTIVE	2024-01-08	108
116	Adam	Razali	NON_ACTIVE	2024-01-09	104
118	Afiq	Danish	ACTIVE	2025-01-10	101
205	Afif	Shaqir	NON_ACTIVE	2024-01-10	103
218	Afiq	Danial	ACTIVE	2025-01-10	107
NULL	NULL	NULL	NULL	NULL	NULL

Query:

```
SELECT * FROM payments
WHERE method IN ('FPX', 'CARD');
```

Display payments that use the method of 'FPX' or 'CARD'

Result:

payment_id	amount	paid_on	method	note	student_id
2	200.00	2024-12-01	CARD	December Deposit	138
3	900.00	2024-12-05	FPX	December Rent	205
7	280.00	2025-01-05	CARD	January Rent	170
8	350.00	2025-01-06	FPX	January Rent	100
10	300.00	2025-02-02	FPX	February Rent	218
NULL	NULL	NULL	NULL	NULL	NULL

Query:

```
SELECT * FROM payments
WHERE (amount > 300 AND NOT method = 'TNG')
OR note = 'December Rent';
```

Filtering query that use the combination of AND, OR and NOT

Result:

payment_id	amount	paid_on	method	note	student_id
1	600.00	2024-12-01	CASH	December Rent	118
3	900.00	2024-12-05	FPX	December Rent	205
4	1000.00	2024-12-10	TNG	December Rent	116
5	1200.00	2024-12-03	CASH	August Rent	98
8	350.00	2025-01-06	FPX	January Rent	100
9	375.00	2025-03-01	CASH	March Deposit	185
NULL	NULL	NULL	NULL	NULL	NULL

Query:

```
SELECT method, COUNT(payment_id) as total_transactions, SUM(amount) as total_amount
FROM payments
GROUP BY method;
```

Aggregate functions: Total number of transactions and amount of payments made by each
method

Result:

method	total_transactions	total_amount
CASH	3	2175.00
CARD	2	480.00
FPX	3	1550.00
TNG	2	1325.00

Query:

```
SELECT UPPER(CONCAT(fname, ' ', lname)) as full_name  
FROM students;
```

String functions: Display students' full name in uppercase

Result:

full_name
RIZWAN HAKIM
ALIF FATHI
NAIM ABDULLAH
ADAM RAZALI
AFIQ DANISH
SYAHMI FARIS
MIKAIL AISY
MUKHRITZ IMAN
AFIF SHAQIR
AFIQ DANIAL

Task 3 - Reporting and Aggregation

Query:

```
#Create view v_room_status
CREATE OR REPLACE VIEW v_room_status AS
SELECT
    r.room_no,
    rt.type_name,
    rt.rent,
    r.floor_no,
    rt.capacity,

    #Count how many ACTIVE students assigned to this room
    (SELECT COUNT(*)
     FROM students s
     WHERE s.room_id = r.room_id
     AND s.status = 'ACTIVE'
    ) AS n_occupants,

    #Count how many maintenance issue are still OPEN for this room
    (SELECT COUNT(*)
     FROM maintenance m
     WHERE m.room_id = r.room_id
     AND m.status = 'OPEN'
    ) AS pending_issues,

    #Room is vacant if no ACTIVE students
    CASE
        WHEN (
            SELECT COUNT(*)
            FROM students s
            WHERE s.room_id = r.room_id
            AND s.status = 'ACTIVE'
        ) = 0
        THEN TRUE
        ELSE FALSE
    END AS is_vacant
FROM rooms r
JOIN room_types rt ON r.type_id = rt.type_id;

select * from v_room_status;
```

Creation of view v_room_status

Result:

	room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
►	A101	Single	600.00	4	1	1	0	0
	B101	Double	400.00	3	2	1	1	0
	C101	Premium	900.00	2	1	0	1	1
	D101	Family	1000.00	1	4	0	0	1
	E101	Twin Share	350.00	5	2	1	0	0
	F101	Deluxe Single	750.00	6	1	1	1	0
	G101	Triple	300.00	1	3	1	1	0
	H101	Executive Suite	1200.00	2	1	0	0	1
	J101	Budget Double	280.00	3	2	1	0	0
	K101	Studio	650.00	3	1	0	0	1

Query:

```
#total number of students per room type
SELECT rt.type_name,
COUNT(s.student_id) AS total_students
FROM room_types rt
JOIN rooms r ON r.type_id = rt.type_id
LEFT JOIN students s ON s.room_id = r.room_id
GROUP BY rt.type_name;
```

Total number of students per room type

Result:

type_name	total_students
Single	1
Double	1
Premium	1
Family	1
Twin Share	1
Deluxe Single	1
Triple	1
Executive Suite	1
Budget Double	1
Studio	1

Query:

```
#average rent & total deposit per room type
SELECT
    type_name,
    ROUND(AVG(rent),2) AS avg_rent,
    ROUND(SUM(deposit), 2) AS total_deposit
FROM room_types
GROUP BY type_name;
```

Average rent and total deposit per room type (rounder to 2 decimal point)

Result:

type_name	avg_rent	total_deposit
Single	600.00	300.00
Double	400.00	200.00
Premium	900.00	450.00
Family	1000.00	500.00
Twin Share	350.00	175.00
Deluxe Single	750.00	375.00
Triple	300.00	150.00
Executive Suite	1200.00	600.00
Budget Double	280.00	140.00
Studio	650.00	325.00

Query:

```
#monthly payment totals grouped by year & month
SELECT
    YEAR(paid_on) AS pay_year,
    MONTH(paid_on) AS pay_month,
    SUM(amount) AS total_paid
FROM payments
GROUP BY YEAR(paid_on), MONTH(paid_on)
ORDER BY pay_year, pay_month;
```

Monthly payment totals grouped by year and month

Result:

pay_year	pay_month	total_paid
2024	12	4225.00
2025	1	630.00
2025	2	300.00
2025	3	375.00

Query:

```
#count OPEN maintenance issues per floor (having > 2)
SELECT
    r.floor_no,
    COUNT(m.maint_id) AS open_issues
FROM rooms r
JOIN maintenance m ON m.room_id = r.room_id
WHERE m.status = 'OPEN'
GROUP BY r.floor_no
HAVING COUNT(m.maint_id) > 2;
```

Count of OPEN maintenance issues per floor (HAVING > 2)

Result:

floor_no	open_issues
----------	-------------

No rows were returned because no floor currently has more than two OPEN maintenance issues in the dataset.

Query:

```
#report queries
SELECT
    rt.type_name,

    # ROUND() example
    ROUND(rt.rent, 2) AS rounded_rent,

    # CONCAT() + UPPER() example
    UPPER(CONCAT(rt.type_name, ' ROOM')) AS type_name_upper,

    # LOWER() example
    LOWER(rt.type_name) AS type_name_lower,

    # CASE example (categorizing rent)
    CASE
        WHEN rt.rent < 400 THEN 'LOW'
        WHEN rt.rent BETWEEN 400 AND 800 THEN 'MEDIUM'
        ELSE 'HIGH'
    END AS rent_category

FROM room_types rt;
```

Combined SQL function report using ROUND(), UPPER(), LOWER(), CONCAT(),
CASE(LOW/MEDIUM/HIGH)

Result:

type_name	rounded_rent	type_name_upper	type_name_lower	rent_category
Single	600.00	SINGLE ROOM	single	MEDIUM
Double	400.00	DOUBLE ROOM	double	MEDIUM
Premium	900.00	PREMIUM ROOM	premium	HIGH
Family	1000.00	FAMILY ROOM	family	HIGH
Twin Share	350.00	TWIN SHARE ROOM	twin share	LOW
Deluxe Single	750.00	DELUXE SINGLE ROOM	deluxe single	MEDIUM
Triple	300.00	TRIPLE ROOM	triple	LOW
Executive Suite	1200.00	EXECUTIVE SUITE ROOM	executive suite	HIGH
Budget Double	280.00	BUDGET DOUBLE ROOM	budget double	LOW
Studio	650.00	STUDIO ROOM	studio	MEDIUM

Summary table

Tables	Number of Records
room_types	10
rooms	10
students	10
maintenance	10
payment	10